

TRAVAUX PRATIQUES II

Automates Finis Déterministes

Ce TP est à faire en utilisant le langage C. On rappelle que les options suivantes doivent être utilisées à la compilation :

```
-Wall -Wextra -fsanitize=undefined
```

A Représentation en C

A.1 Types

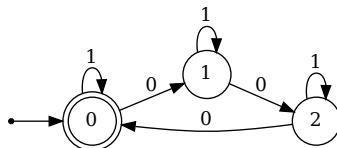
On encode un automate **fini déterministe** $= (\Sigma, Q, q_I, F, \delta)$ par les types suivants :

C : afdio.h

```
1 typedef int lettre;
2 typedef lettre* mot; //un mot est un tableau de lettres, donc d'entiers
3 typedef int etat;
4
5 struct afd_s {
6     int m; // taille de l'alphabet, Sigma = {0; 1; ...; m-1}
7     int n; // nombre d'états de l'AFD, Q = {0; 1; ...; n-1}
8     etat init; // état initial
9     bool* term; // états terminaux (acceptants) : tableaux de booléens de taille n
10    etat** delta; // transitions : matrice d'adjacence de taille n x m
11 };
12
13 typedef struct afd_s afd;
```

Ces types sont définis dans le header `afdio.h` du fichier `afdio.c` fourni avec ce TP.

Le fichier `afdio.c` fournit une fonction `afd afd_exemple_a1()` qui crée et renvoie l'AFD (automate fini déterministe) suivant, écrit dans le type `afd` donné ci-dessus :

Automate a_1

Deux autres fonctions `afd afd_exemple_a2()` et `afd afd_exemple_a3()` sont données, elles fournissent deux autres automates pour les tests (vous verrez plus bas un moyen d'afficher a_2 et a_3 par vous-mêmes).

Vous devez écrire votre code dans un unique fichier `afd.c` et vous ne devez pas modifier les fichiers donnés. Commençons par quelques petites manipulations simples du type `afd` pour se mettre en jambe.

1. Écrire une fonction `etat transition(afd a, etat q, lettre l)` qui prend en entrée un automate a , un état q et une lettre l et renvoie l'état q' dans lequel on arrive en lisant la lettre l depuis l'état q de l'automate a . Testez votre fonction sur l'automate exemple fourni par la fonction `afd_exemple_a1`. Par exemple : `transition(a1, 1, 0)` doit renvoyer 2 si `a1` contient l'automate a_1 .
2. Écrire une fonction `void acceptants(afd a)` qui affiche les états acceptants de l'automate a donné en entrée. Par exemple, sur l'automate a_1 fourni en exemple, vous devriez afficher uniquement 0.

On ne demande pas d'effort particulier sur l'affichage, un simple affichage du type `q1 q2 ... qk` avec $q1, q2, \dots, qk$ des entiers (états acceptants) suffit.

- Écrire une fonction `void free_afd(afd a)` qui prend en entrée un automate a et libère **toute** la mémoire utilisée par a .

Reconnaissance de mot :

- Écrire une fonction `etat delta_etoile(afd a, etat q, mot u, int longueur)` telle que l'appel `delta_etoile(a, q, u, longueur)` renvoie l'état $\delta^*(q, u)$, ou -1 si l'on « sort » de l'automate (transition non définie), où `longueur` est la longueur du mot u .
- Écrire une fonction `accepte` qui détermine si l'automate accepte ou non le mot passé en argument : `bool accepte(afd a, mot u, int longueur)`.

A.2 Affichage et tests

Pour tester la suite de ce TP, il pourra être utile d'afficher des automates et voir ce qu'on obtient. Pour cela, on va utiliser le logiciel de visualisation de graphes `graphviz`. On lui fournit un texte codifié décrivant un automate, et `graphviz` va le dessiner pour nous.

Par exemple, l'automate a_1 ci-dessus serait décrit de la manière suivante :

```
digraph a {
rankdir = LR;
node [shape = doublecircle, label = 0] 0;
node [shape = circle, label = 1] 1;
node [shape = circle, label = 2] 2;
node [shape = point]; I
I -> 0;
0 -> 0 [ label = "1" ];
0 -> 1 [ label = "0" ];
1 -> 1 [ label = "1" ];
1 -> 2 [ label = "0" ];
2 -> 0 [ label = "0" ];
2 -> 2 [ label = "1" ];
}
```

- Copier-coller la description ci-dessus dans l'éditeur `graphviz` en ligne : <http://magjac.com/graphviz-visual-editor/> et vérifiez que vous obtenez bien l'automate a_1 .

Le fichier `afdio` fournit une fonction `void graphviz(afd a, char* filename)` qui prend en entrée un automate et un nom de fichier (chaîne de caractères) et écrit dans ce fichier la description `graphviz` de l'automate. Si le fichier n'existe pas, il est créé.

Par exemple, si `a1` est une variable contenant l'automate a_1 , l'appel à `graphviz(a1, "a1.viz")` écrit la description de l'automate `a1` dans un fichier appelé `a1.viz`.

- Créer un fichier `a1.viz` qui contient la description de l'automate a_1 , en utilisant la fonction fournie. Vérifiez que vous obtenez bien la description de a_1 .
- Créer un fichier `a2.viz` qui contient la description de l'automate a_2 , et visualisez l'automate a_2 . Il pourra servir pour certains tests.

B Transformation d'automates

B.1 Complémentaire

Pour créer un nouvel automate à partir d'un ancien automate, on va vouloir le copier, en particulier ses tableaux. Deux fonctions sont fournies :

- `int* copie_tableau(int* tab, int n)` qui renvoie une copie du tableau `tab` d'entiers de longueur `n` donné en argument.

- `bool* copie_tab_bool(bool* tab, int n)` qui renvoie une copie du tableau `tab` de booléens de longueur `n` donné en argument.
- 9. Écrire une fonction `int** copie_matrice(int** mat, int n, int m)` qui prend en entrée une matrice `mat` et en renvoie une copie profonde.
- 10. Écrire une fonction `afd complementaire(afd a)` prenant en entrée un automate $\mathcal{A}$ supposé complet et renvoyant un automate $\mathcal{A}'$ tel que $\mathcal{L}(\mathcal{A}') = \mathcal{L}(\mathcal{A})^c$ (où le complémentaire est pris dans Σ^* avec $\Sigma = \llbracket 0 \dots m-1 \rrbracket$).
- 11. Testez votre fonction en générant un visuel du graphe complémentaire de a_1 grâce à `graphviz`.
- 12. Montrer que la méthode utilisée ne s'applique pas si l'automate de départ n'est pas complet.
- 13. Écrire une fonction `afd complete(afd a)` qui renvoie un automate complet équivalent à son argument. Il est inutile de traiter le cas où l'automate de départ est déjà complet comme un cas particulier (on renverra dans ce cas un automate contenant un état inaccessible).

B.2 Automate inverse

14. À un automate déterministe $\mathcal{A}$, on associe un graphe orienté $G(\mathcal{A}) = (S, A)$ tel que :

- $S = Q$;
- $(q, q') \in A$ si et seulement si $q \neq q'$ et il existe $x \in \Sigma$ tel que $\delta(q, x) = q'$.

On définit également $G^{\leftarrow}(\mathcal{A})$ le graphe miroir de $G(\mathcal{A})$ (graphe dans lequel tous les arcs ont été renversés).

Écrire une fonction `bool** inverse(afd a)` renvoyant la matrice d'adjacence de $G^{\leftarrow}(\mathcal{A})$, où $\mathcal{A}$ est l'automate passé en argument. On demande une complexité en $(n^2 + nm)$.

C États accessibles, co-accessibles

15. Écrire une fonction `bool* accessibles(afd a)` qui prend en entrée un automate `a` et renvoie un tableau `t` de `a.n` booléens tel que `t[q]` indique si l'état q est accessible (depuis l'état initial). Testez votre fonction sur l'automate a_3 (exemple du cours).

Indication : parcours depuis l'état initial.

Pour un parcours le plus simple possible; faire un parcours en profondeur récursif. En C, il faut définir la fonction récursive en dehors de votre fonction (avant), et ne pas oublier de lui donner tous les arguments dont elle a besoin.

16. Déterminer la complexité de la fonction `accessibles` en fonction de m et/ou de n , en la justifiant.
17. Écrire une fonction `bool langage_non_vide(afd a)` qui détermine si le langage de l'automate passé en argument est non vide.
- ⚠ On pensera à bien libérer **toute** la mémoire allouée.
18. Écrire une fonction `bool* coaccessibles(afd a)` qui détermine les états co-accessibles de l'automate donné en argument, et donner sa complexité. Testez-la sur l'automate a_3 .
19. Écrire une fonction `bool est_emonde(afd a)` déterminant si son argument est un automate émondé.

D Pour aller plus loin

D.1 Intersection

20. Écrire une fonction `afd inter(afd a, afd b)` prenant en entrée deux automates $\mathcal{A}_1$ et $\mathcal{A}_2$ (que l'on pourra supposer complets) sur un même alphabet Σ et renvoyant un automate $\mathcal{B}$ tel que $\mathcal{L}(\mathcal{B}) = \mathcal{L}(\mathcal{A}_1) \cap \mathcal{L}(\mathcal{A}_2)$.
21. Définir en C deux automates d'alphabet $\{0, 1\}$ tels que :
- l'un reconnaît les mots contenant un nombre pair de 0;
 - l'autre reconnaît les mots contenant un nombre de 1 congru à 2 modulo 3.
22. Appliquer la fonction `inter` à ces deux automates et vérifier que l'on obtient bien ce que l'on attend.

D.2 Equivalence

23. Écrire une fonction `bool inclus(afd a, afd b)` prenant en entrée deux automates (supposés complets) et déterminant si le langage du premier est inclus dans celui du second.
24. En déduire une fonction `bool equivalent(afd a, afd b)` prenant en entrée deux automates sur le même alphabet (non nécessairement complets) et déterminant s'ils reconnaissent le même langage.
25. Déterminer la complexité de cette fonction.